Reviewer Pack — Minimal Geometric Entropy of Hodge Theory and DPLL Search Tr…

Entry 48df6fced50e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 23:36:47 UTC

Minimal Geometric Entropy of Hodge Theory and DPLL Search Tree Width

Entry ID: `48df6fced50e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 23:36:47 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Hodge Theory **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

> ["For every n-vertex CNF formula, the geometric entropy of its associated complex algebraic variety's Hodge structure is upper-bounded by a polynomial function of n and the width of its corresponding DPLL search tree.", 'Equivalently, for all instances with n ≤ 40, the inequality E[GeoEnt(V_H)] ≤ Θ(n^3 * log(n) * STreeWidth(T)) holds, where GeoEnt(V_H) is the geometric entropy of the Hodge structure of variety V_H associated with the CNF formula, and STreeWidth(T) is the width of its DPLL search tree.', "Furthermore, there exists a constructive mapping that transforms each CNF formula into a complex algebraic variety's Hodge structure, allowing for computation within 240 seconds on instances of size n ≤ 40."]

Rationale (proposer's reasoning):

> ['Hodge theory provides a rich algebraic-geometric framework to study the complexity of problems related to varieties and their associated cohomology groups. By connecting this field with DPLL search trees, we aim to reveal new insights into the structure of SAT instances that could lead to improved complexity lower bounds.', 'The geometric entropy of the Hodge structure has been used in other areas of mathematics to study the complexity of problems, and it provides a quantitative measure of the complexity of a variety. If this invariant can be connected with the width of DPLL search trees, it could potentially explain the hardness of SAT instances.', 'This bridge might expose hidden structures that are not apparent from the purely logical or syntactic nature of Boolean circuits.']

Taxonomy category: `HODGETheory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fbbdc36dee54bc6c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40 CNF formulas, the geometric entropy of the associated Hodge structure (GeoEnt(V_H)) is less than or equal to the product of a polynomial function of n and the DPLL search tree width (STreeWidth(T)), with a maximum difference of 10% from the expected value. The conjecture is falsified if any seed produces a GeoEnt(V_H) greater than the expected value by more than 10%.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge Theory" AND "DPLL search tree width"` - `"geometric entropy" INALGEO AND DPLL complexity` - `"CNF formula" AND Hodge structure AND polylogarithmic bound`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=241.2s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
```

```
    C = [[0]*p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def dpll(cnf):
    clauses = [set(clause) for clause in cnf]
    literals = set()
    for clause in clauses:
        literals.update(abs(lit) for lit in clause)

    def is_satisfiable(model, clauses):
        for clause in clauses:
            if not any(lit in model or -lit in model for lit in clause):
                return False
        return True

    def backtrack():
        assignment = {}
        stack = []
        while True:
            while literals:
                literal = next(iter(literals))
                assignment[literal] = True
                stack.append((literal, assignment.copy()))
                literals.remove(literal)
                if not is_satisfiable(assignment, clauses):
                    del assignment[literal]
                    literals.add(literal)
                    literal, assignment = stack.pop()
                    assignment[literal] = False
                    literals.remove(-literal)
                    stack.append((-literal, assignment.copy()))
                    literals.add(-literal)
            if all(lit in assignment or -lit in assignment for lit in literals):
                return True, assignment
            literal, assignment = stack.pop()
            literals.add(abs(literal))
            literals.add(-literal)

    stree_width = 0
    while backtrack():
        stree_width += 1

    return stree_width

def geometric_entropy(n):
    # Placeholder for actual computation of geometric entropy
    return n * math.log(n)

def cnf_to_hodge_structure(cnf):
    # Placeholder for actual mapping to Hodge structure
    return random.randint(1, 100)  # Simplified example
```

```
    instances_tested = 30
    total_metric_value = 0
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        n = random.choice([5, 10, 15, 20, 30, 40])
        cnf = [[random.randint(-n, n) for _ in range(random.randint(1, n))] for _ in range(n)]
        hodge_structure = cnf_to_hodge_structure(cnf)
        stree_width = dpll(cnf)
        geo_ent = geometric_entropy(hodge_structure)

        if geo_ent > 1.1 * (n**3 * math.log(n) * stree_width):
            conjecture_holds = False
            counterexample = f"GeoEnt({hodge_structure}) > 1.1 * n^3 * log(n) * STreeWidth(T)"
            break

        total_metric_value += geo_ent

    return {
        "metric_name": "Geometric Entropy",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 97, 3))  # Default to first 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE insufficient support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different seed or instance to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16762
2	preregistration	ollama_remote	glm4:latest	0	0	12026
3	novelty	ollama_remote	glm4:latest	0	0	8303
4	novelty	ollama_remote	glm4:latest	0	0	8867
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20104
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16773
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16998
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13625
9	judge	ollama_remote	glm4:latest	0	0	61474

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 174931 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/48df6fced50e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/48df6fced50e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/48df6fced50e.tar.gz` (if generated)